



DataDriven Floorplanning

ICCAD Problem C

B12901012 張維傑

B12901080 李俊翰

B12901185 謝承恩



Outline

- Input / Output Structure & Modification of the Problem
- RL for floorplanning
- Paper - HouseDiffusion
- Paper - DiffPlace
- Diffusion Model for floorplanning
- Reference



Modification of the Problem

- Two constraints are recategorized to **Hard** instead of **Soft**.
 - Preplaced block
 - Fixed-shaped block



Structure of the Intel Repository

- Evaluator
- Dataset



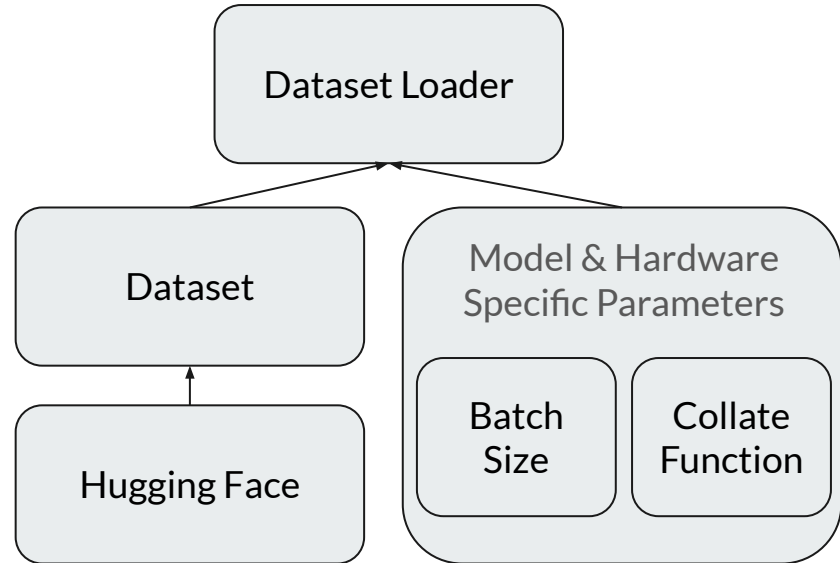
Evaluator - Input/Output Structure

- Submit a python script for evaluation
- We will need to implement `CustomOptimizer.solve()`

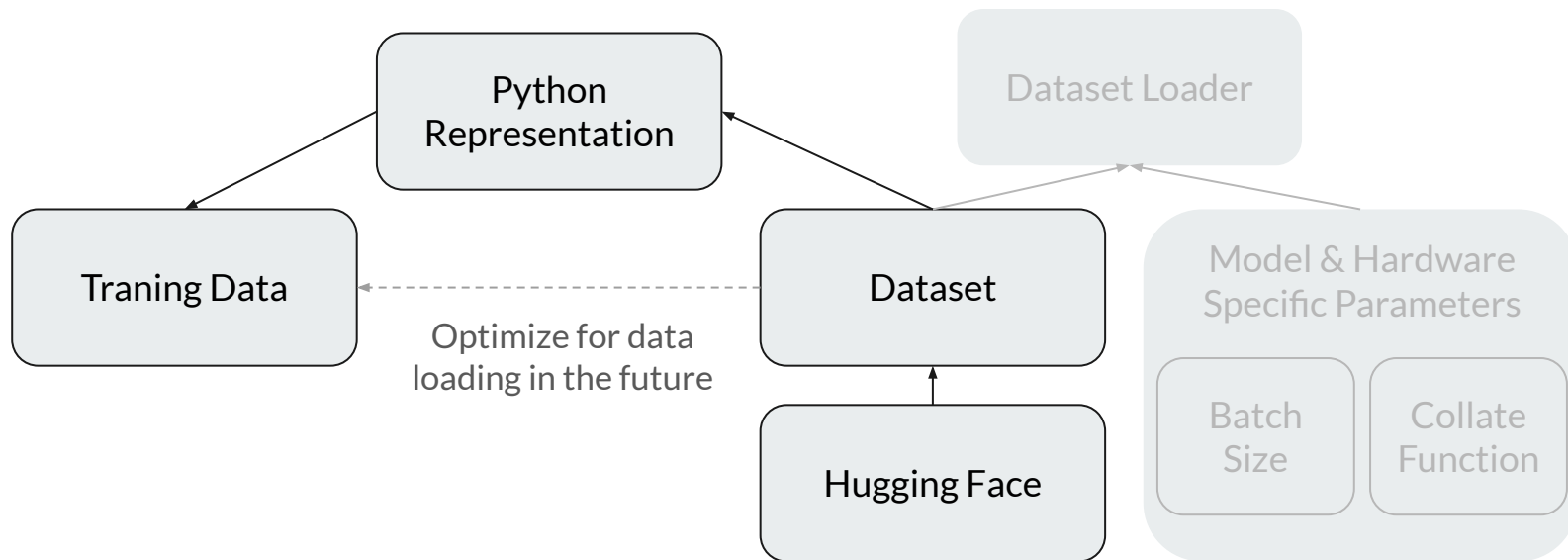
```
def solve(  
    self,  
    block_count: int,  
    area_targets: torch.Tensor,  
    b2b_connectivity: torch.Tensor,  
    p2b_connectivity: torch.Tensor,  
    pins_pos: torch.Tensor,  
    constraints: torch.Tensor,  
    target_positions: torch.Tensor = None  
) -> List[Tuple[float, float, float, float]]:
```

Dataset - Input/Output Structure

- The dataset loader provided by Intel is limited to specific Model & Hardware parameters



Our Dataset Wrapper





Training Data Preprocess

- Data extraction from tensors and ground truth
- Easily accessible by Colab

class DatasetCollection:

- download_train_set() -> None
- download_test_set() -> None
- load_train_single(idx) -> FloorplanInstance
- load_test_single(idx) -> FloorplanInstance
- load_train_list(start, end) -> list
- load_test_list(start, end) -> list
- get_train_data_loader(batch) -> DataLoader

class FloorplanInstance:

- instance_id (int)
- num_blocks (int)
-
- # Raw Inputs (Tensors)
 - area_target
 - b2b_connectivity / p2b_connectivity
 - pins_pos
 - placement_constraints_block
-
- # Ground Truth (Tensors)
 - tree_sol
 - fp_sol
 - metrics_sol
-
- # Processed Constraints (Lists)
 - constr_fixed_size
 - constr_preplaced
 - constr_cluster_peers
 - constr_mib_group_peers
 - constr_boundary



Google Colab Example

Pull from GitHub

```
from google.colab import userdata
token = userdata.get('GITHUB_PAT')

!git clone --recursive https://{token}@github.com/ICCAD2026-Problem-C/floorset-io.git
```

Simple Access

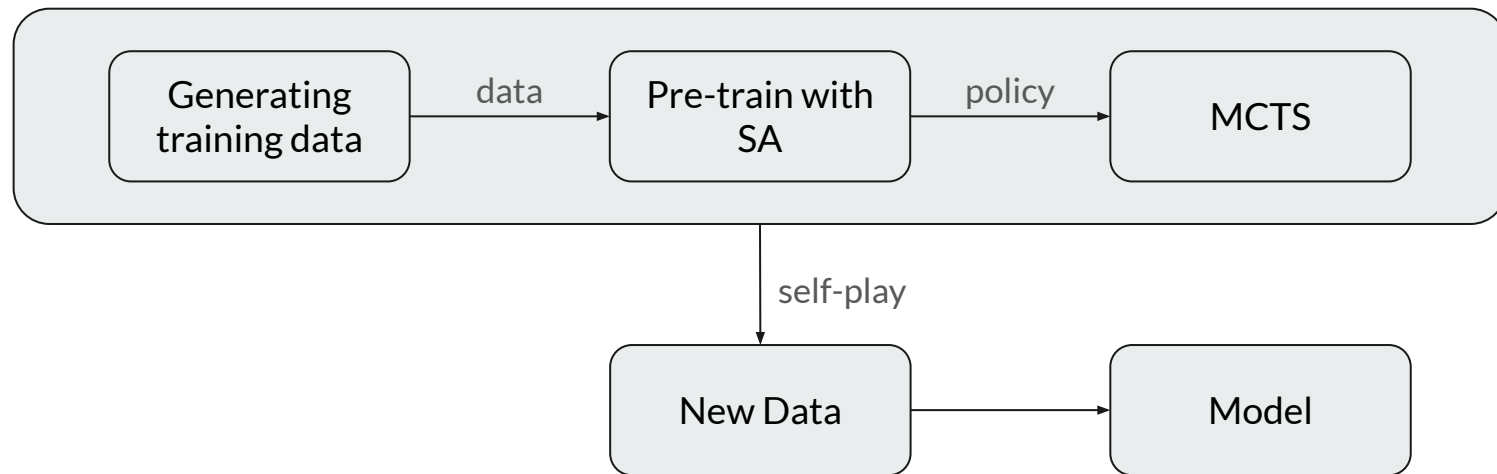
```
import sys
sys.path.append('/content/floorset-io')
from floorset_io import FloorplanInstance, DatasetCollection

collection = DatasetCollection('.')
collection.download_train_set()

instance = collection.load_train_single(0)

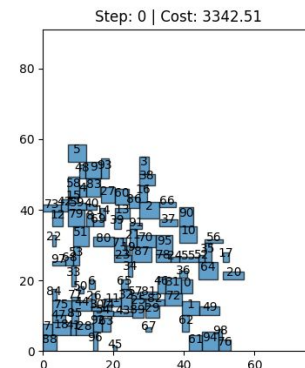
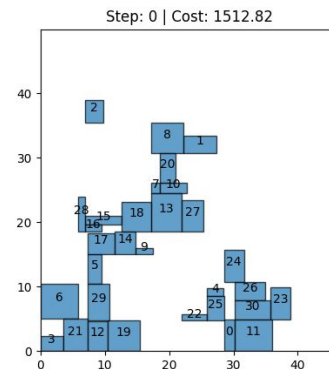
print("Area target:", instance.area_target, sep='\n')
```

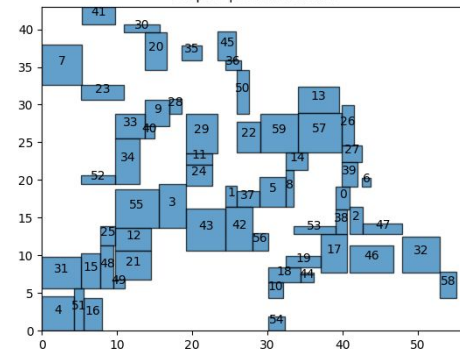
RL for Floorplanning



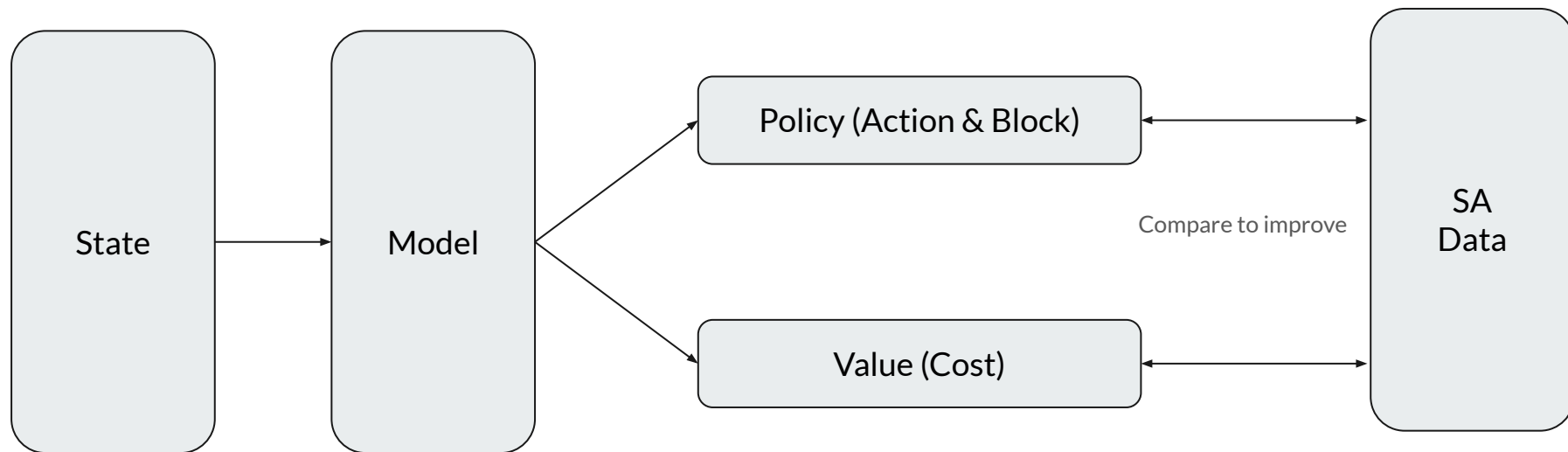
RL for Floorplanning - Generating Data

- Generating training data for pre-train
- **SA** is used to save the good solution for training
- Data
 - num_block: 30 - 120
 - width, height: 1.0 - 6.0 (float with .2f)
 - Only considering the area (i.e. no HPWL / penalty)



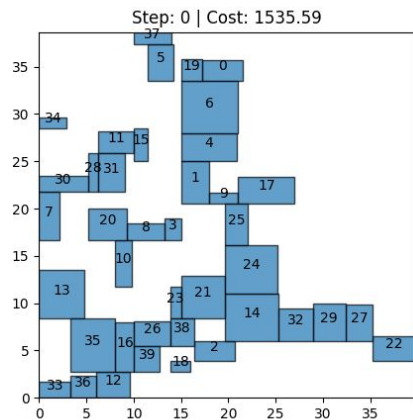


RL for Floorplanning - Pretrain



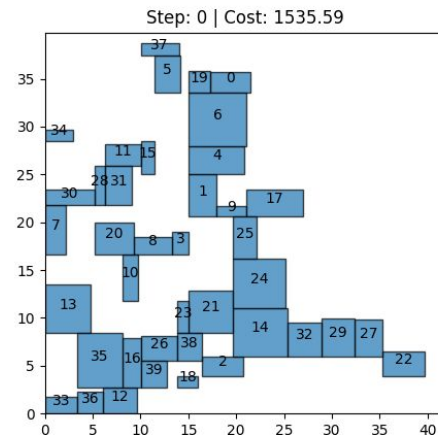
RL for Floorplanning - MCTS

- **Selection:** Select a node by ucb_score, then along the path to execute
- **Evaluation:** When encountering an unseen state, call neural network to evaluate
- **Expansion:** Expand the most promise node
- **Backpropagation:** Propagate back to update data



RL for Floorplanning - SelfPlay

- Combine the **pretrain model** and **MCTS** to selfplay
- Generate selfplay data to modify the pretrain model
- Self-play for 50 rounds with each round goes 100 step -> cost 5 hours to train





RL for Floorplanning - Result

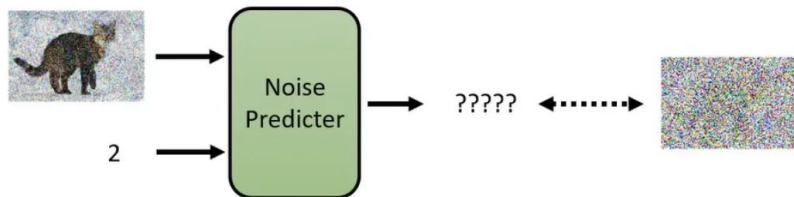
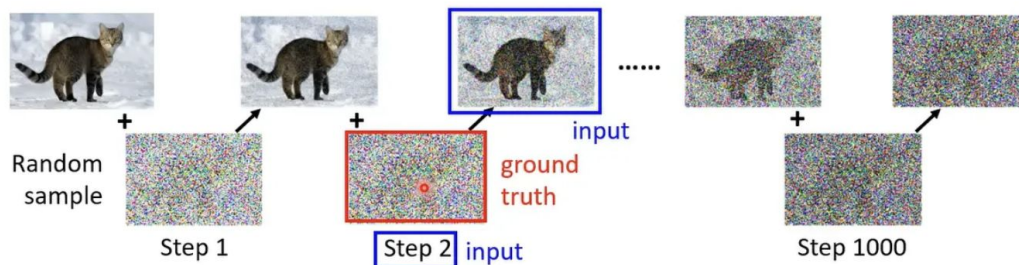
- Not enable to do efficient floorplanning in few steps
- Possible reasons:
 - Lack of training data
 - MCTS thinks not deep enough



Diffusion Model

- Thermodynamic Inspiration
- Bidirectional Markov Chain: The overall framework consists of two opposing, step-by-step stochastic processes.
- Forward Process (Diffusion): A fixed, parameter-free process that incrementally adds Gaussian noise to the original data until it degrades into pure, isotropic noise (complete disorder).
- Reverse Process (Denoising): Trains a neural network to iteratively "denoise" the data starting from pure noise, step-by-step reconstructing the joint probability distribution of the original data.

Diffusion Model



資料來源

: <https://www.youtube.com/watch?v=azBugJzmz-o>

Forward Process

- Step-by-Step Noise Injection: The forward process $q(x_t | x_{t-1})$ is fixed, adding a small amount of Gaussian noise at each step controlled by a predefined variance schedule β_t .

Training

repeat

$x_0 \sim q(x_0)$ ← Sample 出一張 x_0 乾淨的圖

$t \sim \text{Uniform}(\{1, \dots, T\})$

$\epsilon \sim \text{Normal}(0, I)$ ← 從 normal distribution (mean: 0, variance: 1 for every dimension)

$\nabla_{\theta} \| \epsilon - \epsilon_{\theta}(\sqrt{\alpha_t} x_0 + \sqrt{1 - \alpha_t} \epsilon, t) \|^2$ ← $\sqrt{\alpha_t} x_0 + \sqrt{1 - \alpha_t} \epsilon$ add noise α_t 為最大, α_T 為最小 (Noisy Image)
 $\epsilon_{\theta}(\text{noisy image}, t)$ 為一個 function (network)
 ϵ : Target Noise → 學習的目標

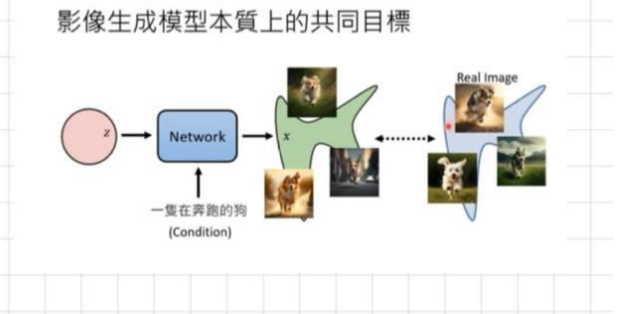
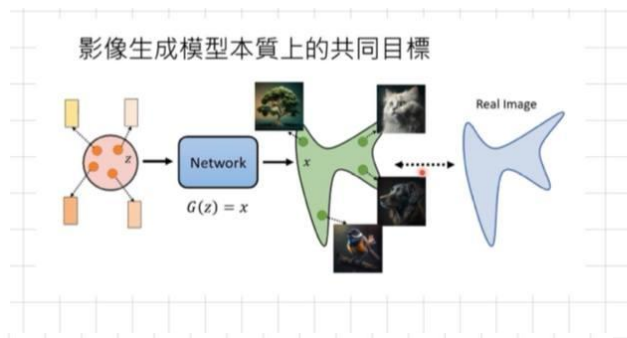
The Reverse Process and Prediction Objective

Sampling

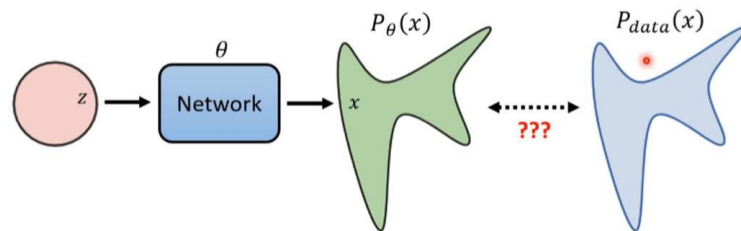
```
 $x_T \sim N(0, I)$   
for  $t = T, \dots, 1$  do  
   $z \sim N(0, I)$  if  $t > 1$ , else  $z = 0$   
   $x_{t-1} = \frac{1}{\sqrt{\alpha_t}} (x_t - \frac{1-\alpha_t}{\sqrt{1-\alpha_t}} \epsilon_\theta(x_t, t)) + \alpha_t z$   
end for  
return  $x_0$ 
```

noise predictor

Essence of Diffusion Model



Maximum Likelihood Estimation



Sample $\{x^1, x^2, \dots, x^m\}$ from $P_{data}(x)$

We can compute $P_\theta(x^i)$

???

$$\theta^* = \arg \max_{\theta} \prod_{i=1}^m P_\theta(x^i)$$

從訓練資料 $P_{data}(x)$ 中取樣



House Diffusion: Vector Floorplan Generation via a Diffusion Model

- Research Task: Automatically generating architectural floorplans using a bubble diagram as the input constraint.
- Core Contribution: Proposing a novel Diffusion Model approach to directly generate vector-graphics floorplans, eliminating the need for complex post-processing.
- Breaking Limitations: Overcoming the lack of global reasoning inherent in traditional raster (image pixel) generation methods.



1D Polygonal Loops

- Geometric Representation: represent a floorplan as a set of 1D polygonal loops instead of traditional binary image representation
- Structural Definition: Each loop corresponds to a specific room or door and is defined by a sequence of 2D corner coordinates.
- Coordinate Transformation: These corner coordinates are treated as continuous values to be mixed with Gaussian noise in the forward process, while simultaneously represented as discrete binary integers in the reverse process.



Continuous and Discrete Dual-Track Denoising

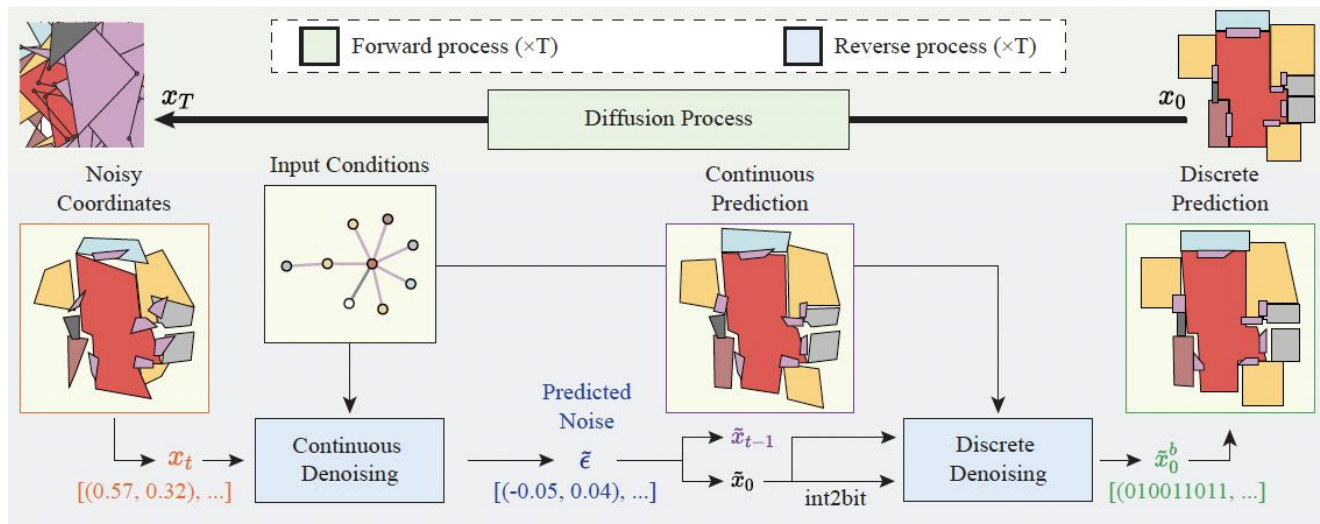
- Geometric Challenge: Direct regression of coordinates struggles to achieve precise geometric incident relationships, such as wall alignment or corner-sharing.
- Continuous Denoising: The model infers a single-step noise amount as a continuous quantity to precisely invert the continuous forward diffusion process.
- Discrete Denoising: The model simultaneously infers the final 2D coordinates as discrete quantities to establish precise geometric relationships like parallelism, orthogonality, and corner-sharing.

Continuous and Discrete Dual-Track Denoising

```
1 [Sampling 過程]
2
3 x_T ~ N(0, I) // 1. 從純高斯雜訊開始 (連續的浮點數座標)
4 conditions // 2. 氣泡圖條件 (決定哪些房間相連)
5
6 for t = T down to 1 do
7
8     // =====
9     // 第一階段：連續去噪 (Continuous Denoising)
10    // 【Attention 發生的位置一】在這裡的 network_continuous 內部
11    // =====
12    epsilon_pred, x_0_pred = network_continuous(x_t, t, conditions)
13
14    // 套用標準 DDPM 公式，推導出這一步的連續座標 x_{t-1}
15    z ~ N(0, I) if t > 1 else z = 0
16    x_t_minus_1_cont = (1 / sqrt(alpha_t)) *
17    (x_t - ((1 - alpha_t) / sqrt(1 - alpha_bar_t)) *
18    epsilon_pred) + sigma_t * z
19
```

```
21 // =====
22 // 第二階段：離散去噪 (Discrete Denoising)
23 // =====
24 if t < 32 then
25     // 1. 將連續預測出的最終座標 x_0_pred，轉換為 8-bit 的二進位離散表示
26     x_0_bit = int2bit( round( denormalize(x_0_pred) ) )
27
28     // 2. 讓二進位特徵進入離散分支的神經網路
29     // 【Attention 發生的位置二】在這裡的 network_discrete 內部
30     x_0_bit_pred = network_discrete(x_0_bit, t, conditions)
31
32     // 3. 經過閾值處理 (0或1)，並轉回十進位整數，得到絕對對齊的整數座標
33     x_0_int_pred = bit2int( threshold(x_0_bit_pred) )
34
35     // 4. 用這個「被鎖死對齊」的整數座標，重新加噪到 t-1 時刻，
36     // 作為下一輪迴圖的輸入，強制網路接受幾何約束
37     x_t_minus_1 = add_noise(normalize(x_0_int_pred), t-1)
38 else
39     // 如果 t >= 32，只用連續座標繼續走
40     x_t_minus_1 = x_t_minus_1_cont
41 end if
42
43 x_t = x_t_minus_1
44 end for
45
46 // 最終輸出最後一步離散網路預測並轉換的絕對整數座標
47 return x_0_int_pred
```

Continuous and Discrete Dual-Track Denoising





Transformer-Based Attention Mechanism

- Transformer-based neural network architecture
- Global Self Attention (GSA): Processes the relationships between every pair of corners across all rooms in the floorplan.
- Component-wise Self Attention (CSA): Limits attention to nodes within the same room or door to ensure logical and valid shapes.
- Relational Cross Attention (RCA): Focuses specifically on the relationships between connected rooms and doors, guided by the input bubble diagram constraint.



Conclusion

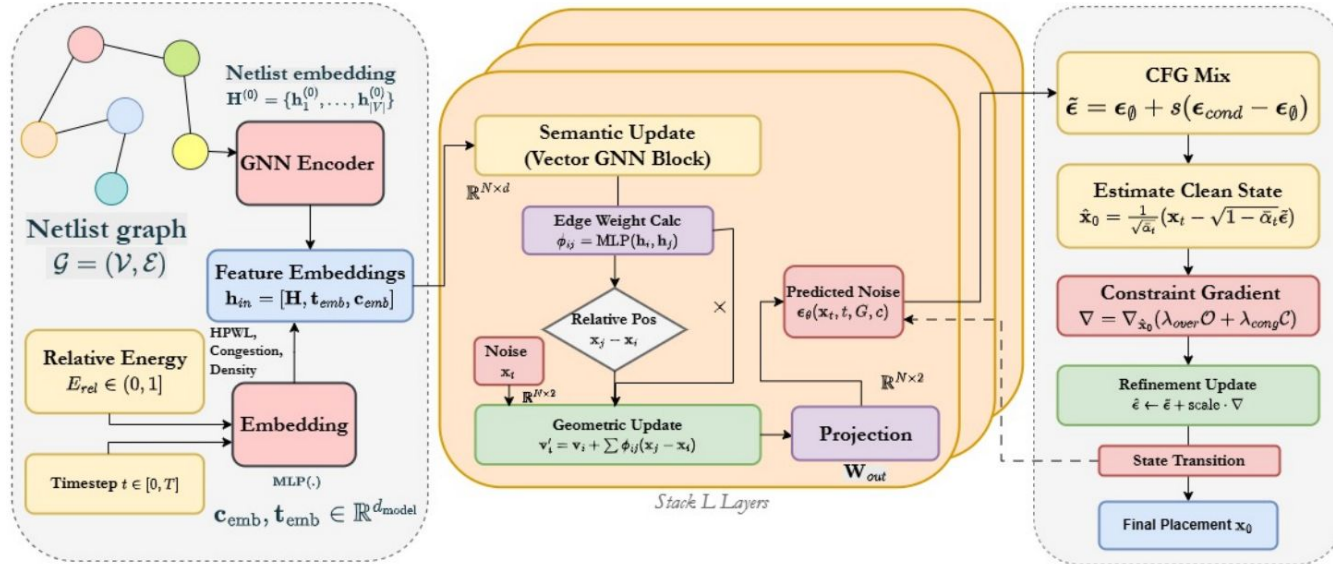
- Superior Performance: Better than House-GAN++
- Non-Manhattan Support
- Precise Detail Control: Allows users to explicitly specify and control the exact number of corners per room, offering highly customizable design options.
- Ideas: multiple attention mechanism & discrete denoising



DiffPlace - Introduction

- A diffusion based placement tool for macro placement
- Fixed-sized block placement
- Optimize for HPWL, congestion, and overlaps

DiffPlace - Framework

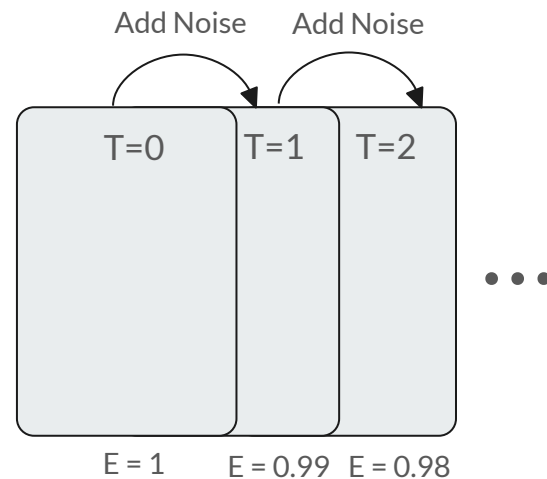


DiffPlace - “Energy”

- Higher energy = Higher quality placement (lower HPWL, less overlap)
- Normalized to (0, 1] to adjust for scale difference
- Set E_{rel} to 1 for Inference

$$E(\mathbf{x}, G) = \lambda_{\text{HPWL}} \frac{\text{HPWL}(\mathbf{x})}{L_{\text{norm}}} + \lambda_{\text{cong}} \text{RUDY}(\mathbf{x}, G) + \lambda_{\text{over}} \text{Overlap}(\mathbf{x})$$

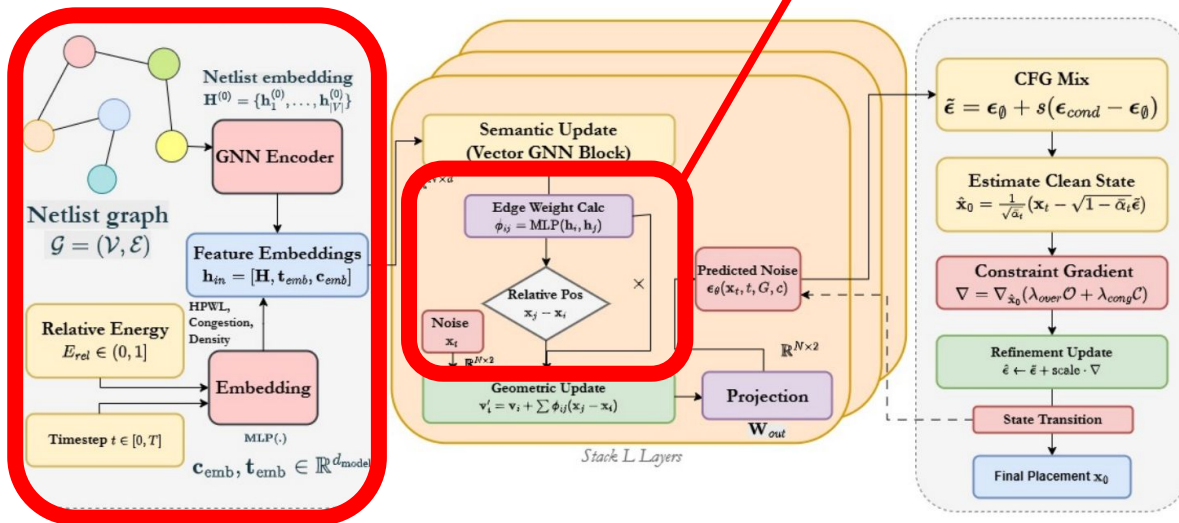
$$E_{\text{rel}}(\mathbf{x}, G) = \exp \left(-\kappa \cdot \frac{E(\mathbf{x}, G) - E_{\text{ana}}(G)}{E_{\text{ana}}(G)} \right).$$



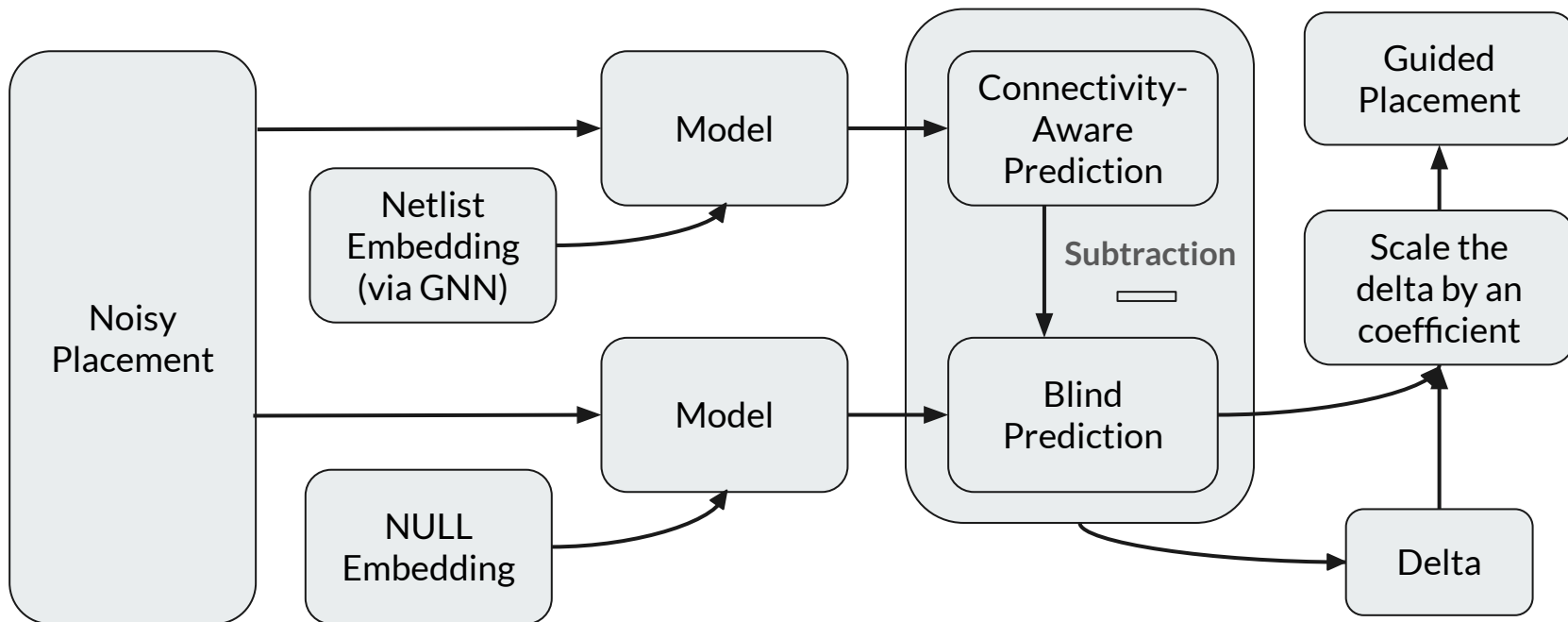
DiffPlace - GNN Encoding

Generate netlist embedding via GNN

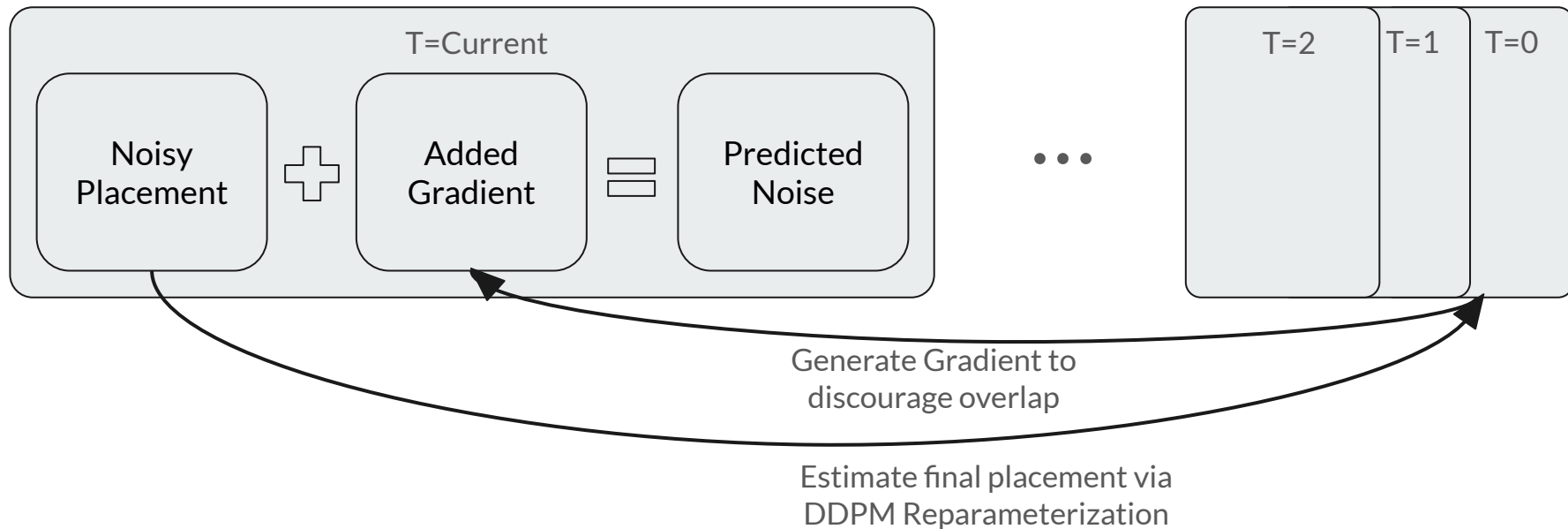
Relative Position is processed by GNN



DiffPlace - Classifier Free Guidance



DiffPlace - Manifold Constraint Guidance



DiffPlace - Placement Result

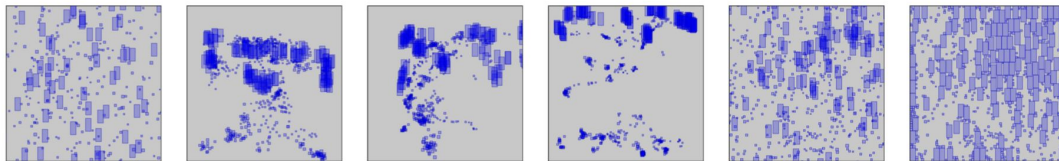


Fig. 1: Visualizing the progressive denoising process on the `ariane133` benchmark. DiffPlace simultaneously places both SRAM macros and clustered standard-cell blocks. From left to right: The process initializes from Gaussian noise $x_T \sim \mathcal{N}(0, I)$ in normalized coordinates and iteratively refines the layout. Intermediate panels display the model's clean estimate $\hat{x}_0$ at selected diffusion steps, culminating in the final placement x_0 (rightmost panel).

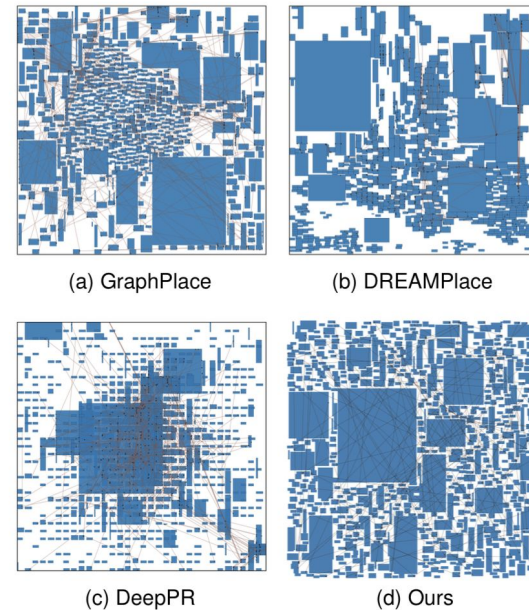


Fig. 4: Visualization of placement results on `adaptec3` benchmark: (a) GraphPlace, (b) DeepPR (c) DREAMPlace, and (d) Ours. DiffPlace produces more compact placements with shorter wirelength compared to existing approaches.

DiffPlace - Placement Result

TABLE III: Comparison of Macro-Only HPWL ($\times 10^5$) on IBM Benchmarks. **Bold** indicates the best result, and underlined indicates the second-best.

Circuit	MaskPlace	WireMask-BBO	ChiPFormer	DiffPlace (Ours)
ibm01	4.30	2.78	3.88	<u>3.15</u>
ibm02	5.54	4.19	5.05	<u>4.45</u>
ibm03	3.31	3.30	3.74	<u>3.45</u>
ibm04	6.91	5.43	5.96	<u>5.60</u>
ibm06	0.93	<u>0.85</u>	0.87	0.82
ibm07	2.67	<u>2.66</u>	2.36	<u>2.48</u>
ibm08	20.60	19.20	19.90	<u>19.45</u>
ibm09	2.45	1.76	<u>1.77</u>	1.95
ibm10	23.80	18.20	18.25	<u>18.22</u>
ibm11	4.15	3.75	3.25	3.60
ibm12	14.90	<u>11.80</u>	13.00	11.50
ibm13	4.58	4.41	4.02	4.25
ibm14	8.43	9.80	7.44	<u>7.80</u>
ibm15	4.68	7.77	2.67	4.10
ibm16	18.30	<u>14.80</u>	15.50	14.50
ibm17	16.80	12.20	13.70	13.10
ibm18	5.98	3.44	4.19	3.90
Avg.	8.72	7.43	7.79	<u>7.60</u>

TABLE IV: Comparison of Macro-Only HPWL ($\times 10^5$) on TILOS Benchmarks. “Failed” indicates the method failed to converge or generate a legal solution within 3 hours. **Bold** indicates the best result.

Benchmark	DREAMPlace	MaskPlace	WireMaskBBO	ChiPFormer	DiffPlace (Ours)
ariane133	15.20	16.85	17.10	16.50	12.71
ariane136	18.10	19.42	–	–	15.44
mempool	0.12	–	Failed	–	0.09
nvda	32.40	Failed	Failed	Failed	25.71

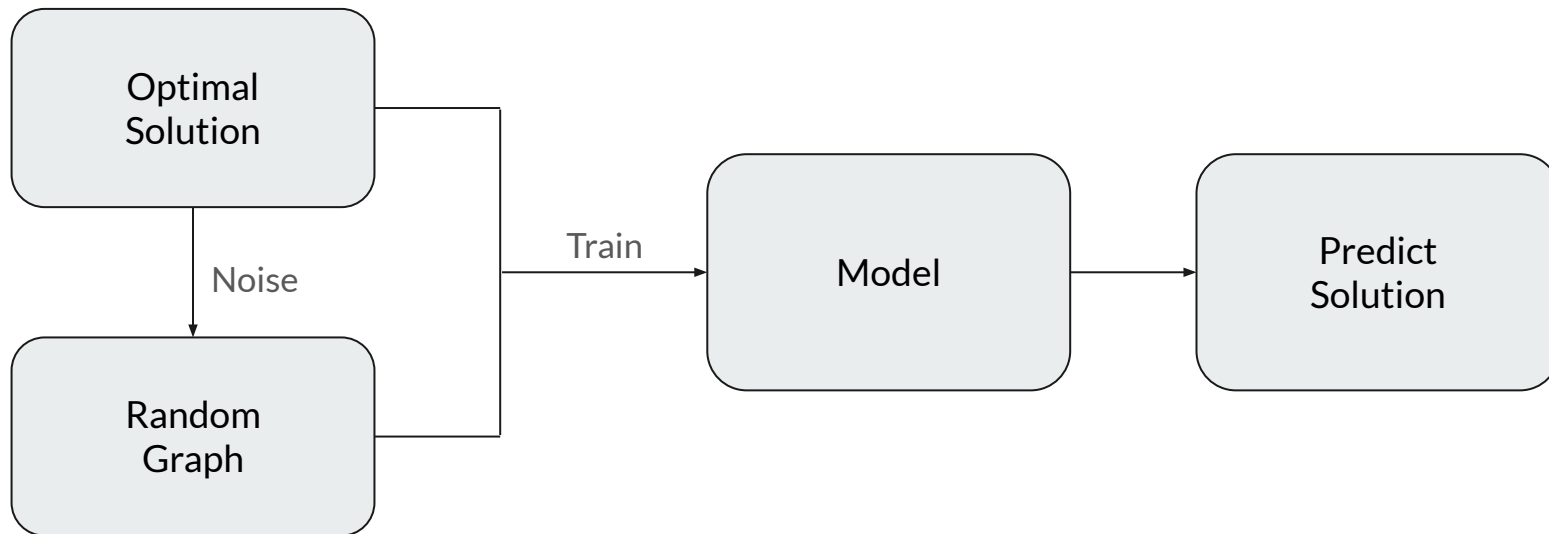
DiffPlace - Conclusion

- Unlike RL-based method, do not require online training
- Linear time complexity: $O(T \cdot (N_{\text{nodes}} + N_{\text{edges}}))$
- Runtime comparison: Dreamplace < Diffplace < Analytical & RL-based methods

TABLE XIII: Comparison of runtime (minutes) for macro placement on the IBM benchmark. Circuit `ibm05` is omitted as it contains no macros.

Method	ibm01	ibm02	ibm03	ibm04	ibm06	ibm07	ibm08	ibm09	ibm10	ibm11	ibm12	ibm13	ibm14	ibm15	ibm16	ibm17	ibm18
MaskPlace	151.0	162.0	120.0	66.0	36.0	60.0	72.0	53.0	512.0	81.0	385.0	91.0	390.0	85.0	104.0	482.0	63.0
WireMask-BBO	205.0	201.0	220.0	211.0	221.0	220.0	209.0	218.0	230.0	222.0	250.0	228.0	264.0	251.0	219.0	263.0	214.0
ChiPFormer	100.0	89.0	73.0	80.0	82.0	83.0	103.0	70.0	241.0	104.0	199.0	125.0	190.0	110.0	135.0	254.0	91.0
EfficientPlace	52.0	64.0	60.0	63.0	30.0	61.0	81.0	49.0	265.0	82.0	202.0	93.0	214.0	88.0	128.0	360.0	56.0
DREAMPlace	0.31	0.41	0.39	0.40	0.23	0.26	0.26	0.26	0.46	0.30	0.47	0.61	0.76	0.92	0.78	0.84	0.74
DiffPlace (Ours)	2.59	3.15	3.04	3.09	3.33	4.02	4.73	4.31	11.25	8.50	13.10	10.48	14.88	17.03	19.70	25.83	20.15

Diffusion Model for Floorplanning





Diffusion Model for Floorplanning - Noise

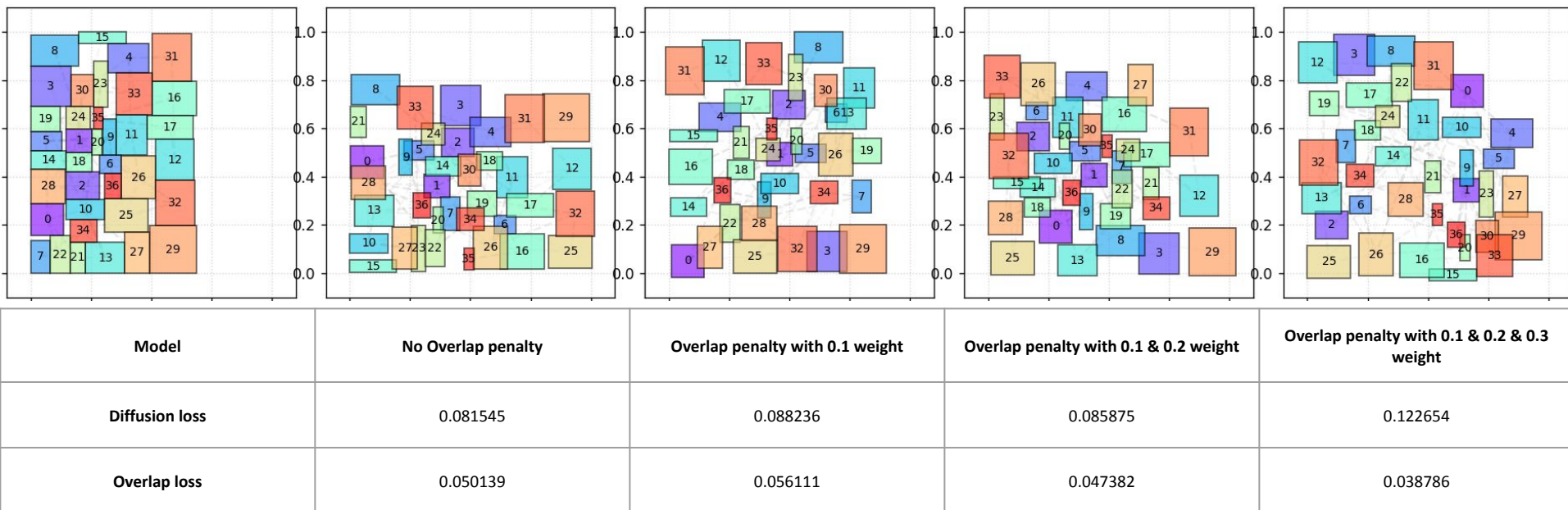
- Optimal Solution: Generated by **SA**, with 10,000 datasets
 - **Blocks information:** x_coord, y_coord, width, height
 - **Edge information:** block 1, block 2, weight
- Add noise for 1,000 times, with DDPM equation $X_t = \sqrt{\bar{\alpha}}X_0 + \sqrt{1 - \bar{\alpha}}\varepsilon$



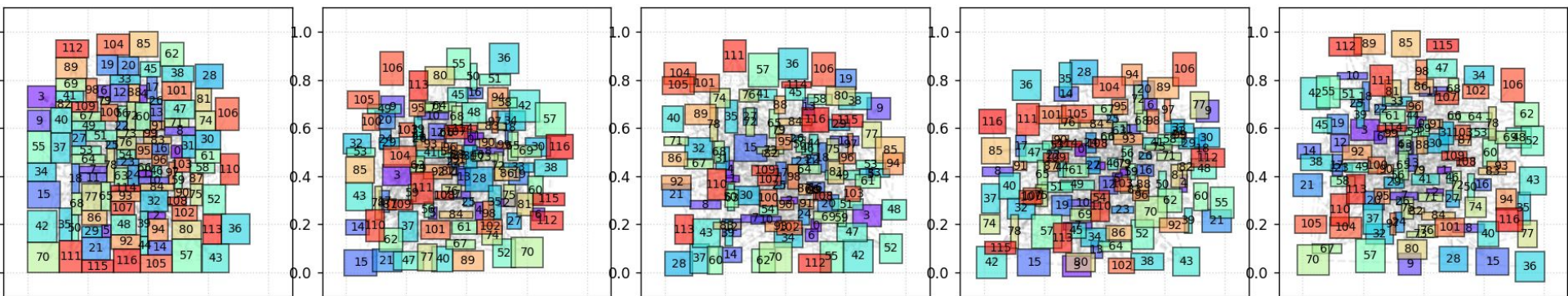
Diffusion Model for Floorplanning - Training

- Trying to get the original figure from the noise figure
 -  Predict the noise and calculate the loss with the noise added
 -  Directly compare the graph with the original graph
- Dealing with overlap
 - Calculate the overlap area and modify the loss
 - Adjust the weight of overlap loss to make a balance of diffusion loss and overlap

Diffusion Model for Floorplanning - Predict



Diffusion Model for Floorplanning - Predict



Model	No Overlap penalty	Overlap penalty with 0.1 weight	Overlap penalty with 0.1 & 0.2 weight	Overlap penalty with 0.1 & 0.2 & 0.3 weight
Diffusion loss	0.098640	0.103709	0.097277	0.087711
Overlap loss	0.158350	0.126513	0.143803	0.106286



Diffusion Model for Floorplanning - TODO

- Adjust the weight of overlap penalty to strike a balance
- Deal with the overlap
 - Force-Directed Algorithm
 - Analytic Placement
 - Legalization



Reference

- [1] Qi Xu, Hao Geng, Song Chen, Bo Yuan, Cheng Zhuo, Yi Kang, Xiaoqing Wen (2021, Nov. 30). *GoodFloorplan: Graph Convolutional Network and Reinforcement Learning-Based Floorplanning*.
- [2] H. Ren and J. Hu, Eds., Machine Learning Applications in Electronic Design Automation. Cham, Switzerland: Springer Nature Switzerland AG, 2022.
- [3] A. Mirhoseini, A. Goldie, M. Yazgan, et al., "A graph placement methodology for fast chip design," Nature, vol. 594, no. 7863, pp. 359–364, 2021.
- [4] Trung, K. L., & Hy, T. S. (2025). DiffPlace: A Conditional Diffusion Framework for Simultaneous VLSI Placement Beyond Sequential Paradigms. *arXiv preprint arXiv:2510.15897*.
- [5] [5] Shabani, M. A., Hosseini, S., & Furukawa, Y. (2022) . House Diffusion: Vector Floorplan Generation via a Diffusion Model with Discrete and Continuous Denoising . *arXiv preprint arXiv:2211.13287*.